Does "Reproduce the Bug First" Help Coding Agents?

Natural Experiments in 353,009 Agent Trajectories

Rajan Yadav
Independent researcher
rajanyadav0307@gmail.com

Abstract

Advice for AI coding agents often says to reproduce a bug with a test script before fixing it, and to re-run the script afterwards. Studies that compare successful and failed agent runs tend to support such advice, but these comparisons mix easy and hard issues and can mislead. We use public trajectory datasets in which the same agent attempted the same issue many times. Because the input is identical across attempts, differences in what the agent does first arise from sampling randomness, which yields natural experiments. Comparing attempts only within the same issue, we find that reproducing first *lowered* the success of an older Llama-70B SWE-agent by 3.5 points per 100 attempts (95% CI −4.8 to −2.2; 1,048 issues). The mechanism is visible in the traces: reproducing first used up context and delayed reaching the file that needed the fix (median step 7 vs. 2). For 4 newer models in 7 configurations, testing first made no meaningful difference (+0.1 points; 90% CI −0.4 to +0.5, inside a pre-registered ±2-point margin). Simple pooled comparisons of the same data are badly misleading: across 57 tip–AI pairs they differ from the within-issue estimates by 10.9 points on average, and per-AI simple estimates of up to +27.1 points shrink to −1.0 to +1.5. Re-running one's own test after a fix ("check your fix") is associated with modest gains for two of eight AI configurations. We release the full pipeline, which reproduces every number in this paper.

CCS Concepts

• **Software and its engineering** → **Software testing and debugging**; • **Computing methodologies** → *Artificial intelligence*; • **Mathematics of computing** → *Probability and statistics.*

Keywords

coding agents, SWE-bench, trajectories, natural experiments, confounding, Simpson's paradox

1 Introduction

Software-engineering agents such as SWE-agent [24] and OpenHands [23] resolve repository issues by running a loop of tool calls. A large body of practical advice tells these agents how to work: reproduce the bug first, run the tests, check the fix, clean up. Agent designs bake the advice in [7], and analyses of trajectories report that runs which reproduce or validate are more often successful [17, 25].

Whether such advice *causes* success is a different question. Issues differ greatly in difficulty, and agents behave differently on easy and hard issues. If an agent writes a reproduction script mostly on issues that are easy to reproduce, and those issues are also easy to fix, a pooled comparison will credit the script for the issue's easiness (Figure 1). This is the classic problem of confounding and of Simpson's paradox [22]. Recent work shows that pooling across tasks inflates behavior–outcome associations in agent trajectories and that some published effects vanish or reverse once the task is held fixed [15].

We exploit a feature of public trajectory datasets: the same agent, with the same prompt and the same issue text, was run many times on each issue. With nothing in the input to tell attempts apart, the agent's choice of first action varies only through sampling. For the older SWE-agent model, about 41% of attempts begin by creating a script and the rest begin by searching the code, often on the very same issue on the same day. Comparing these two groups *within* each issue is a natural experiment.

Our contributions are:

- **A natural-experiment design for agent logs.** We compare repeated attempts within an issue, control for a hidden mixture of model versions using the evaluation day, and check the design with balance tests, a placebo, and exact-history "forks".
- **Reproducing first hurt an older agent and did not help newer ones.** The older model lost 3.5 points. For 4 newer models in 7 configurations the effect is equivalent to zero within ±2 points, in every leave-one-AI-out analysis.
- **The mechanism.** Reproducing first cost the older model about 10.0 extra steps, made it run out of context +11.0 points more often, and delayed localization of the gold file.
- **The size of the simple-analysis trap.** Across 57 tip–AI pairs, pooled and within-issue estimates differ by 10.9 points on average and point in opposite directions in 37% of pairs.

The hypotheses, estimators and decision rules were frozen before any confirmatory run.

2 Related Work

Trajectory analyses. Many studies relate agent behavior to success: action patterns and behavioral profiles [9], behavioral drivers across agents [17], and the role of reproduction tests in repair [6]. These are observational and compare runs across tasks. Khanna [15] shows that such pooled comparisons are inflated by between-task structure and recommends within-task comparisons and permutation calibration. We adopt that recommendation, add a version control and coin-flip framing, and measure named tips across eight AI configurations.

Identification from sampling noise. He et al. [12] note that, given a history, the next action is randomized by decoding noise, and they estimate step effects by *replaying* continuations from prefixes. We use the same logic without replay, relying on attempts that already exist in public logs; at our scale, population-level effects are well powered. Sam-Bodden [20] reports that even temperature-0

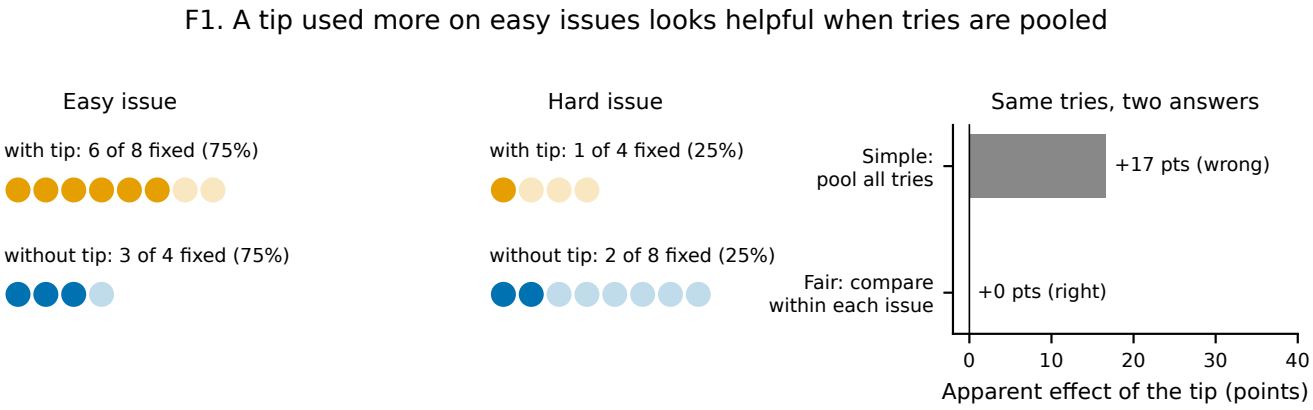


Figure 1: Why pooled comparisons mislead. Within each issue the tip changes nothing, but it is used more on the easy issue, so pooling all attempts makes it look helpful. Our estimator compares attempts within an issue and then averages.

inference flips about 9% of per-instance outcomes, which supports the premise that repeated attempts differ by chance.

Interventions on instructions. Prompt studies find that repository context files push agents toward more exploration and testing without higher success [11], and that an instruction only to reproduce before patching does not help and can hurt [10]. Our results agree, using natural variation on real issues rather than modified tasks.

3 Data

We use three public datasets (Table 1 lists their Hugging Face IDs), each pinned to a Hugging Face revision with SHA-256 checksums.

- **Older AI** [18]: SWE-agent runs of the `swe-agent-llama-70b` model on SWE-bench-style issues [13], with a median of 13 attempts per issue, with evaluation timestamps.
- **Nebius newer AI** [19]: OpenHands runs of Qwen3-Coder on SWE-rebench issues [3]. It keeps every attempt, including failures.
- **NVIDIA newer AIs** [1]: Open-SWE-Traces. We use the 7 subsets (4 models across SWE-agent and OpenHands, two task sources) that have repeated attempts per issue and a shared editing tool.

Cleaning. We converted all 402,043 attempts and 26.5M steps into one schema (Table 3). We kept attempts whose issue text matches the most common text for that issue (so inputs are identical), dropped NVIDIA attempts with unknown outcome, and, for the older AI, kept attempts restricted to the target model. NVIDIA had already removed attempts that hit the step limit or produced no patch [1]; Section 7 discusses this.

Hidden model versions. The older dataset's single model label covers several versions: success rates by evaluation day range from 2% to 50%, and the share of attempts that start by writing a script moves with them (Figure 2). We therefore compare attempts within issue *and* evaluation day.

Labels. A shared rule-based labeler maps every action to one of twelve kinds (e.g. create reproduction script, edit code, run, run test

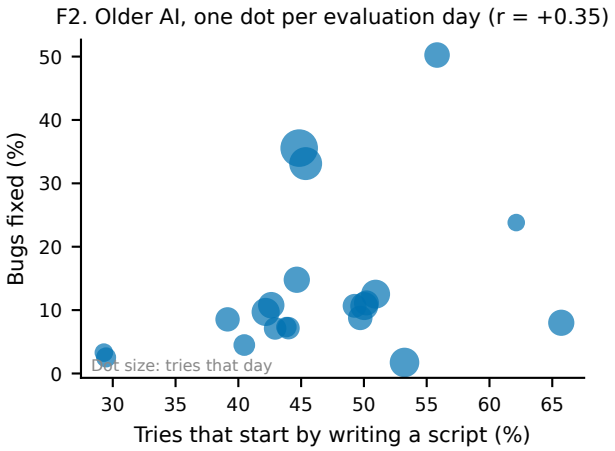


Figure 2: Older AI by evaluation day: success and the share of attempts that start by writing a script move together, revealing a mixture of model versions.

suite, look). We validated it in two ways. First, we checked each label against the tool's reply (e.g. a "create" was confirmed by the file-created message). Second, a blind second labeling of stratified samples agreed with the rules with Cohen's κ [8] of 0.90 ($n = 645$) and, on a fresh sample, 0.86 ($n = 300$). The reproduction-script label has precision 100% and recall 73%; we test the effect of missed scripts in Section 6.

4 Fair-Comparison Method

Estimator. For a binary choice a vs. b (e.g. reproduce first vs. search first) we form strata: issue × day for the older AI, and AI configuration × issue for the newer AIs. In stratum s with n_{as} and

Table 1: Datasets as analysed (attempts with identical input and known outcome).

Dataset	Source (revision)	Tries	Issues	Tries per issue (median)	Steps	Bugs fixed
Older AI (SWE-agent 2024, Llama-70B)	nebius/SWE-agent-trajectories (68195a1)	73,738	3,591	13	1,933,044	16.4%
Nebius OpenHands (Qwen3-Coder)	nebius/SWE-rebench-openhands-trajectories (3545538)	67,049	6,306	10	4,316,962	48.0%
NVIDIA Open-SWE-Traces (4 AIs, 7 configurations)	nvidia/Open-SWE-Traces (1db6741)	212,222	99,018	2	20,230,229	45.2%

n_{bs} attempts and success rates $\hat{p}_{as}$, $\hat{p}_{bs}$, the estimate is the Mantel–Haenszel-weighted [16] average of within-stratum differences,

$$\hat{\Delta} = \frac{\sum_s w_s(\hat{p}_{as} - \hat{p}_{bs})}{\sum_s w_s}, \qquad w_s = \frac{n_{as} n_{bs}}{n_{as} + n_{bs}},$$

using only strata that contain both groups. We report 95% (and 90%) CIs from 2,000 bootstrap resamples of whole issues, and a permutation p-value from 1,000 shuffles of group labels within strata. As cross-checks we fit a linear probability model with stratum fixed effects and issue-clustered standard errors [2, 5], and a conditional logistic regression. The *simple* estimate is the pooled difference in success rates with no strata.

Hypotheses. **P1**: for the older AI, the first action is `create` (group R) vs. a search or navigation command (group S). **P2**: for the newer AIs, the attempt creates a test or reproduction file before its first edit of project code ("test first") vs. edits code first ("fix first"). "No meaningful difference" is claimed only if the 90% CI lies within ±2 points (two one-sided tests [21]). **P3**: the mean absolute gap between simple and fair estimates, over all tip–AI pairs, exceeds 2 points. Thirteen secondary analyses (73 estimates) are corrected together with Benjamini–Hochberg [4] at $q = 0.05$. Comparisons with fewer than 100 informative strata are labelled underpowered and not interpreted.

What is and is not a coin flip. P1 is a decision at the first step, made on identical input, so within an issue and day it is as good as random (we test this below). P2 and the after-fix tips (S1–S6) are decisions made mid-run, after the agent has seen some output. For these, within-issue comparisons remove issue difficulty but not everything the agent observed, so we call them same-issue associations. For P2 we add a stricter check: pairs of attempts with *identical* action histories up to the point where one creates a test file and the other does something else.

Design checks. (i) Within issue × day, the evaluation hour does not predict the first move (permutation $p = 0.44$, 1,736 groups). (ii) A placebo comparing two names for the same action, `reproduce.py` vs. `reproduce_bug.py`, gives −0.8 points (95% CI −2.3 to +0.8). (iii) We verify that each AI is "undecided" about the tip, choosing it in between 5% and 95% of attempts; test-first rates in the NVIDIA configurations range from 10% to 98%, while the Nebius Qwen3-Coder tests first in 99% of attempts and therefore offers no natural experiment for P2.

5 Results

Table 2 summarises the primary results.

5.1 Reproducing first hurt the older AI (P1)

Within issue and day, attempts that began by creating a script resolved 3.5 fewer issues per 100 (95% CI −4.8 to −2.2; permutation $p < 0.001$; 1,870 groups, 1,048 issues, 13,509 attempts). The fixed-effects regression gives −3.5 points and the conditional logit an odds ratio of 0.61. The effect is stable across five definitions of the version batch (−2.8 to −4.0; Figure 3).

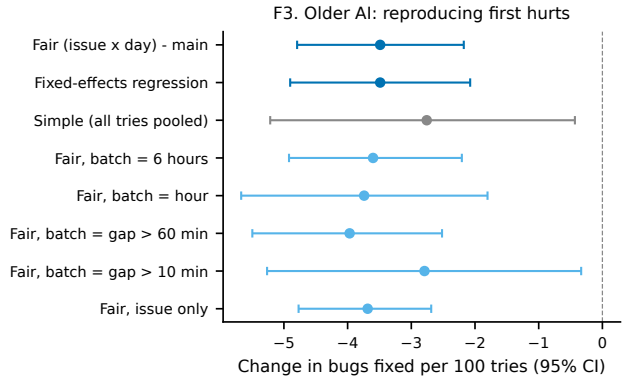


Figure 3: P1 under the main design, the alternative estimators, and alternative version-batch definitions. All 95% CIs exclude zero.

5.2 Why: context and localization

Reproducing first was costly. Within the same issue and day, it added 10.0 steps per attempt and raised the share of attempts that ran out of context by +11.0 points (Figure 4a). It also delayed localization: the median step at which the agent first opened a file changed by the reference fix was 7 for reproduce-first attempts and 2 for search-first attempts (Kaplan–Meier [14]; Figure 4b). A discrete-time hazard model puts the per-step chance of reaching the gold file −12.2 points lower. An agent with a small context window pays for the script with the room it needs to find and fix the bug.

5.3 Testing first made no difference for newer AIs (P2)

Pooled over the 7 newer configurations, testing first changed success by +0.1 points (95% CI −0.5 to +0.6; 90% CI −0.4 to +0.5; 9,776 issues), which is equivalent to zero within ±2 points. Per-configuration estimates range from −1.0 to +1.5 (Figure 5). We found 152,058 pairs of attempts with identical action histories; in the 7,126 attempts where, at the fork, one creates a test file and the other does something else, creating the test file changed success by 0.0 points (95% CI −1.0 to +1.0). The conditional logit odds ratio is 1.02. For the Nebius Qwen3-Coder, which almost always tests first, the one available coin flip at the first step (`think` vs. `view`) also shows no effect (−0.1 points; 95% CI −0.7 to +0.6).

Table 2: Primary estimates, in bugs fixed per 100 attempts (95% CI). Groups / issues / attempts refer to the fair comparison.

Hypothesis	Fair (95% CI)	Permutation p	FE regression	Simple	Groups / issues / tries	Decision
P1 Reproduce first (older AI)	-3.5 [-4.8, -2.2]	<0.001	-3.5 [-4.9, -2.1]	-2.8 [-5.2, -0.4]	1,870 / 1,048 / 13,509	harm
P2 Test first (newer AIs, pooled)	+0.1 [-0.5, +0.6]	0.803	+0.1 [-0.6, +0.8]	+1.9 [+1.2, +2.5]	9,776 / 9,776 / 25,773	equivalent within ±2 points
P3 Mean |simple − fair| over tip × AI pairs	10.9 [10.3, 11.9]	—	—	—	57 pairs	supported (> 2 pts)

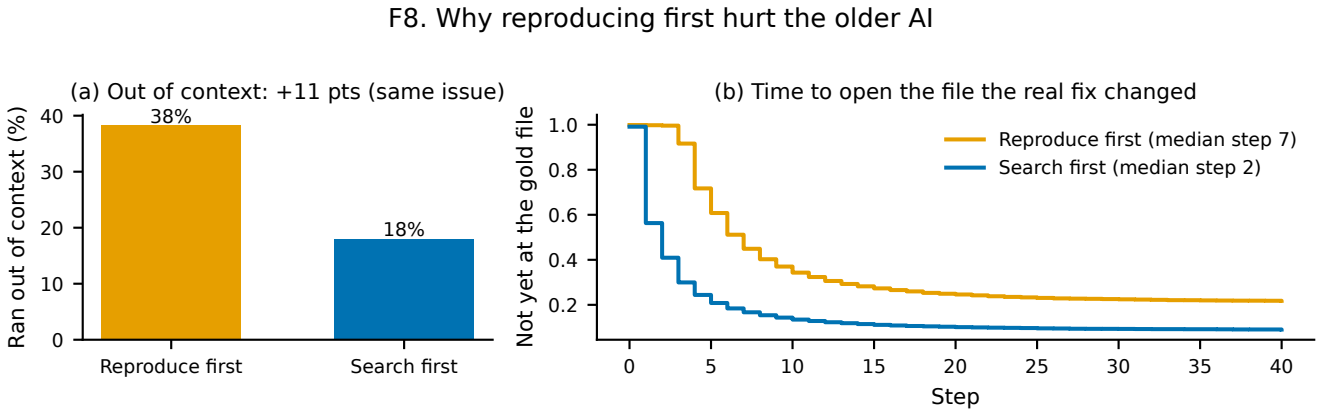


Figure 4: Mechanism for the older AI. (a) Share of attempts that ran out of context. (b) Time to first open a file changed by the reference fix; attempts that never open one are censored.

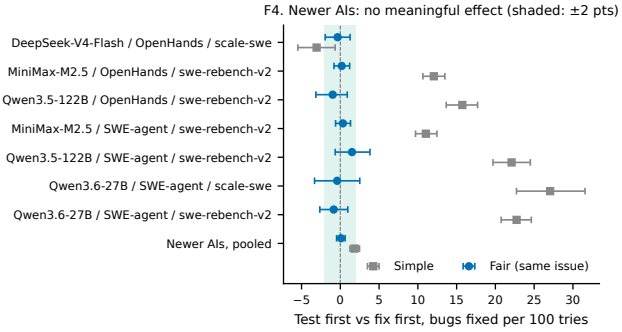


Figure 5: P2 per AI configuration and pooled. Simple pooled estimates (grey) suggest large benefits; within-issue estimates (blue) are all close to zero. Shaded: the pre-registered ±2-point equivalence margin.

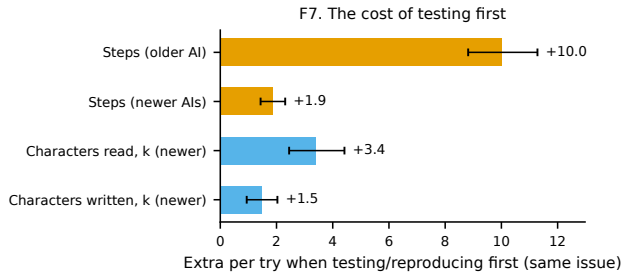


Figure 6: Extra cost per attempt of testing or reproducing first, within the same issue.

Testing first is not free for newer AIs either: it costs 1.9 more steps, 3.4k more characters read and 1.5k more written per attempt (Figure 6). A plausible but untested explanation for the different outcome is that newer models have larger context windows, so the extra cost does not turn into failures.

5.4 Simple analyses mislead (P3)

For every tip–AI pair we computed both the simple pooled difference and the fair within-issue difference (Figure 7). The mean absolute gap is 10.9 points (95% CI 10.3 to 11.9) over 57 pairs; the two point in opposite directions in 37% of pairs and disagree on statistical significance in 77%. Leaving out the underpowered pairs gives 9.7 points over 50 pairs. The P2 data also show Simpson's paradox: pooled across AIs, the simple estimate is +1.9 points, while the per-AI simple estimates range from −3.1 to +27.1, and the fair estimates from −1.0 to +1.5. A study that compared successful and failed runs on this data would conclude that testing first adds up to +27.1 points for some AIs; within the same issue, it adds nothing measurable.

5.5 Other tips

Of 73 secondary estimates, 11 remain significant after correction (Figure 8; full list in Table 4). Re-running one's own test or reproduction script after the first fix ("check your fix") is associated with +2.7 points for the older AI (95% CI +1.9 to +3.6) and +4.1 points for Qwen3.5-122B with OpenHands; its CI includes zero for the other six configurations. Running the project's test suite, writing extra

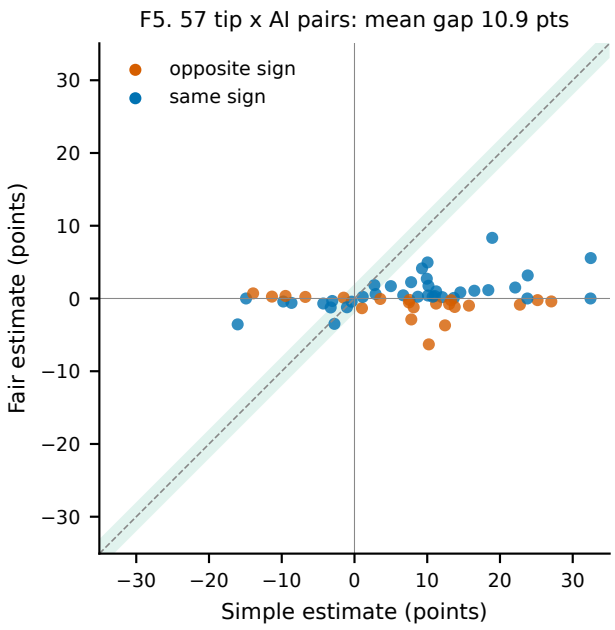


Figure 7: Simple vs. fair estimates for every tip–AI pair. Points far from the diagonal are pairs where a pooled analysis misleads; red points flip sign. Shaded band: within 2 points.

tests, and cleaning up show no consistent association. For the older AI, the choice among search commands as a first move makes little difference (−1.2 to +5.0 points vs. `search_dir`). No interaction with issue type or programming language survived correction.

6 Robustness

Version batches and undated attempts. Some older-AI attempts lack an evaluation time; all of them failed. Adding them back to their issue in proportion to other failures gives −3.1 points; the most adverse possible placement, one attempt at a time, gives −1.4 points.

Leave one AI out. Dropping any one newer configuration keeps every 90% CI for P2 within −0.6 to +0.7 points.

Label errors. A missed reproduction script can only move an attempt from “fix first” to “test first” if the attempt created some file before its first fix. Moving all such attempts gives +0.4 points (95% CI −0.2 to +0.9).

Reproducibility. Running the full analysis twice from scratch yields identical result tables.

7 Threats to Validity

NVIDIA’s filtering. NVIDIA removed attempts that hit the step limit or produced no patch. These are hidden failures. If test-first attempts lost 5.2% more of them than fix-first attempts, P2 would reach −2 points. We calibrated this with the unfiltered Nebius data: there, the share of attempts that such filtering would remove is −2.3 points lower for test-first than fix-first attempts (95% CI −5.4 to +0.6; 273 issues), so the filter, if anything, works against testing first. The calibration comes from a different AI and few issues, so this remains the main limitation of P2.

Mid-run decisions. P2 and S1–S6 are not decided at the first step. Within-issue comparisons remove issue difficulty but not what the agent saw before deciding. The exact-fork analysis addresses this for P2; for S1–S6 we make no causal claim.

Version proxy. The evaluation day is a proxy for the older model’s version. Finer batches (hours, bursts separated by 10- or 60-minute gaps) give the same answer.

Labels and AI-assisted validation. The second labeling used in the label validation was performed with the help of an AI assistant rather than a human annotator, blind to the rule labels. We disclose this and publish both labelings; the primary evidence for label quality is the check against tool replies, which is independent of any annotator.

Generalization. The older AI is a 2024 open model; the newer AIs are recent open models. Results may differ for proprietary agents, other scaffolds, or tasks without hidden tests.

8 Discussion

For an older, context-limited agent, reproducing the bug first was harmful: the script used up space and time the agent needed to find the bug. For newer agents, testing first neither helped nor hurt, at a measurable cost in steps and text. “Check your fix” is the only tip with a positive association, and only for some AIs. A practical reading is that process advice should be evaluated per agent, and that its cost matters when context is scarce.

The larger lesson is methodological. Pooled comparisons of successful and failed runs, the most common analysis of agent trajectories, were off by 10.9 points on average in our data, often in the wrong direction. Repeated attempts of the same issue are already common in public datasets; comparing within them is cheap and much more reliable.

9 Conclusion

Using natural experiments in repeated agent attempts, we found that the popular advice to reproduce a bug first hurt an older coding agent and made no difference for 4 newer ones, while simple analyses of the same data suggest large benefits. We recommend within-issue comparisons as the default for analysing agent trajectories.

10 Data and Code Availability

All datasets are public on Hugging Face; the exact revisions and checksums are listed in the repository. The code (MIT license), including the cleaning, labeling, estimation, robustness, figure and table steps, is available at https://github.com/rajanyadav0307/tips_agentic_ai. The command `agent-tips run-all` reproduces every number, figure and table in this paper from the raw data. The analysis plan was frozen in the repository (tag `analysis-plan-v1`) before the confirmatory runs.

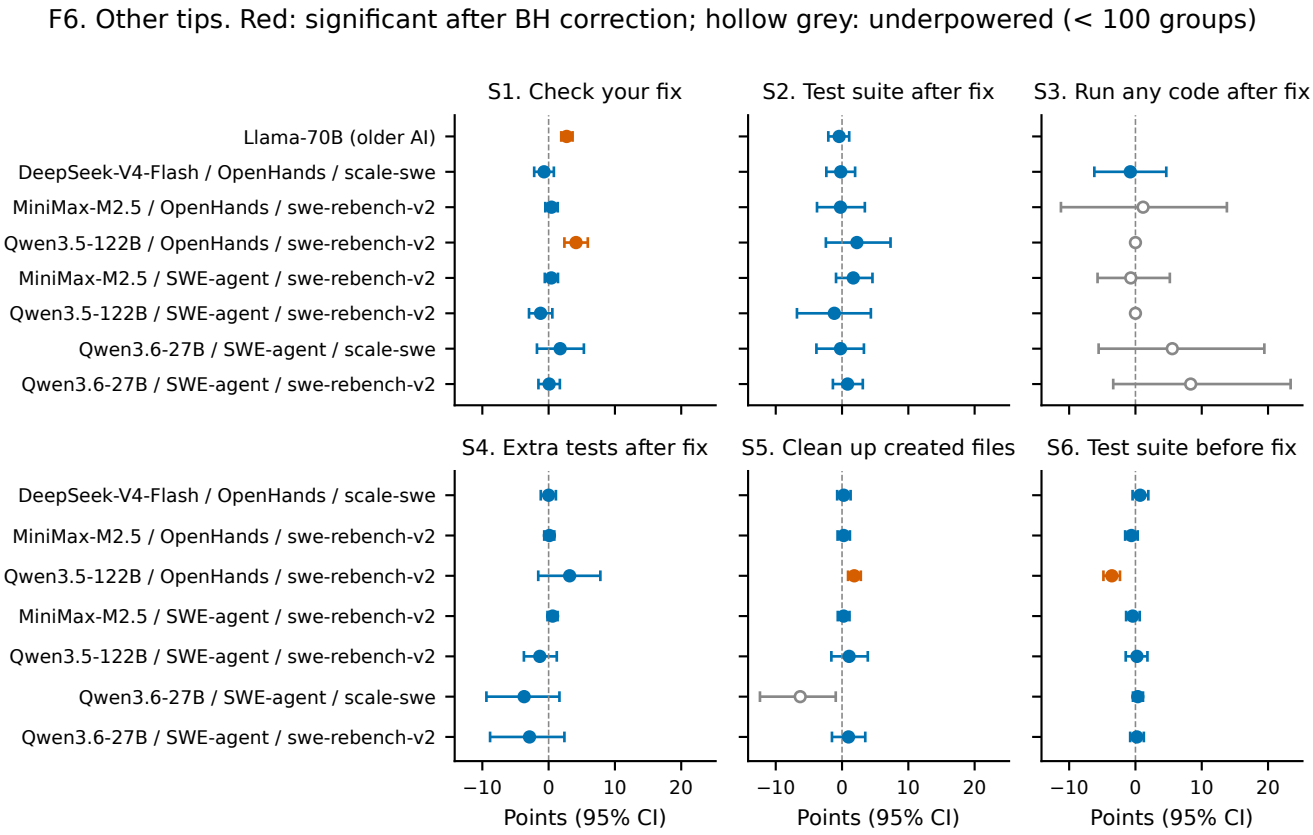


Figure 8: Other tips, fair within-issue estimates per AI configuration. These are same-issue associations, not coin flips.

Use of generative AI. AI coding and writing assistants were used to implement the pipeline, perform the second labeling for label validation (Section 7), and draft this manuscript. The author reviewed all code, results and text and takes responsibility for them.

References

[1] Wasi Uddin Ahmad, Nikolai Ludwig, Somshubra Majumdar, and Boris Ginsburg. 2026. Open-SWE-Traces: Advancing Dual-Mode Multilingual Distillation for Software Engineering Agents. *arXiv preprint arXiv:2606.16038* (2026).

[2] Joshua D. Angrist and Jörn-Steffen Pischke. 2009. *Mostly Harmless Econometrics: An Empiricist's Companion.* Princeton University Press.

[3] Ibragim Badertdinov, Alexander Golubev, Maksim Nekrashevich, Anton Shevtsov, Simon Karasik, et al. 2025. SWE-rebench: An Automated Pipeline for Task Collection and Decontaminated Evaluation of Software Engineering Agents. *arXiv preprint arXiv:2505.20411* (2025).

[4] Yoav Benjamini and Yosef Hochberg. 1995. Controlling the False Discovery Rate: A Practical and Powerful Approach to Multiple Testing. *Journal of the Royal Statistical Society: Series B* 57, 1 (1995), 289–300.

[5] A. Colin Cameron and Douglas L. Miller. 2015. A Practitioner's Guide to Cluster-Robust Inference. *Journal of Human Resources* 50, 2 (2015), 317–372.

[6] Ira Ceka, Hailie Mitchell, Saurabh Pujar, Luca Buratti, Shyam Ramji, et al. 2025. Understanding Automated Program Repair Agents Through the Lens of Traceability: An Empirical Study. *arXiv preprint arXiv:2506.08311* (2025).

[7] Dong Chen, Shaoxin Lin, Muhan Zeng, Daoguang Zan, Jian-Gang Wang, et al. 2024. CodeR: Issue Resolving with Multi-Agent and Task Graphs. *arXiv preprint arXiv:2406.01304* (2024).

[8] Jacob Cohen. 1960. A Coefficient of Agreement for Nominal Scales. *Educational and Psychological Measurement* 20, 1 (1960), 37–46.

[9] Jie Gao, Kaiser Sun, Jen-tse Huang, Katherine Van Koevering, Sijie Ji, et al. 2026. How to Interpret Agent Behavior. *arXiv preprint arXiv:2605.13625* (2026).

[10] Thibaud Gloaguen, Niels Mündler, Mark Müller, Veselin Raychev, and Martin Vechev. 2026. Coding Agents Don't Know When to Act. *arXiv preprint arXiv:2605.07769* (2026).

[11] Thibaud Gloaguen, Niels Mündler-Sasahara, Mark Niklas Müller, Veselin Raychev, and Martin Vechev. 2026. Evaluating AGENTS.md: Are Repository-Level Context Files Helpful for Coding Agents? *arXiv preprint arXiv:2602.11988* (2026).

[12] Jiawei He, Mengyu Shi, Jie Jia, Xikai Yang, and Dong Sun. 2026. What Process Evaluation of Coding Agents Actually Measures: Action, Task, and Step Are Three Different Levels. *arXiv preprint arXiv:2608.22960* (2026).

[13] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. SWE-bench: Can Language Models Resolve Real-World GitHub Issues?. In *International Conference on Learning Representations (ICLR).*

[14] Edward L. Kaplan and Paul Meier. 1958. Nonparametric Estimation from Incomplete Observations. *J. Amer. Statist. Assoc.* 53, 282 (1958), 457–481.

[15] Khanna. 2026. Same Task, Different Fate: Population Structure in Trajectory-Based Analyses of Coding Agents. In *SWGeno '26.* doi:10.1145/3844178.3844642

[16] Nathan Mantel and William Haenszel. 1959. Statistical Aspects of the Analysis of Data from Retrospective Studies of Disease. *Journal of the National Cancer Institute* 22, 4 (1959), 719–748.

[17] Tural Mehtiyev and Wesley Assunção. 2026. Beyond Resolution Rates: Behavioral Drivers of Coding Agent Success and Failure. *arXiv preprint arXiv:2604.02547* (2026).

[18] Nebius. 2024. SWE-agent-trajectories. Hugging Face dataset `nebius/SWE-agent-trajectories`, revision 68195a1.

[19] Nebius. 2025. SWE-rebench-openhands-trajectories. Hugging Face dataset `nebius/SWE-rebench-openhands-trajectories`, revision 3545538.

[20] Brian Sam-Bodden. 2026. What Context Does a Coding Agent Actually Need to Act? *arXiv preprint arXiv:2607.09691* (2026).

[21] Donald J. Schuirmann. 1987. A Comparison of the Two One-Sided Tests Procedure and the Power Approach for Assessing the Equivalence of Average Bioavailability. *Journal of Pharmacokinetics and Biopharmaceutics* 15, 6 (1987), 657–680.

[22] Edward H. Simpson. 1951. The Interpretation of Interaction in Contingency Tables. *Journal of the Royal Statistical Society: Series B* 13, 2 (1951), 238–241.
[23] Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, et al. 2025. OpenHands: An Open Platform for AI Software Developers as Generalist Agents. In *International Conference on Learning Representations (ICLR)*.
[24] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. In *Advances in Neural Information Processing Systems (NeurIPS)*.
[25] Daoguang Zan, Zhirong Huang, Wei Liu, Hanwu Chen, Linhao Zhang, et al. 2025. Multi-SWE-bench: A Multilingual Benchmark for Issue Resolving. *arXiv preprint arXiv:2504.02605* (2025).

A Cleaning funnel

B All secondary estimates

Table 3: Cleaning funnel per dataset.

Dataset	Stage	Tries	Issues	Removed	Bugs fixed
Older AI	All tries in the dataset	80,036	3,591	—	16.7%
Older AI	In scope (swe-agent-llama-70b)	74,792	3,591	5,244	16.7%
Older AI	Same issue text as the most common one	73,738	3,591	1,054	16.4%
Older AI	Dated (evaluation timestamp found)	63,269	3,490	10,469	19.1%
Nebius OpenHands	All tries in the dataset	67,074	6,306	—	47.9%
Nebius OpenHands	Same issue text as the most common one	67,049	6,306	25	47.9%
NVIDIA Open-SWE-Traces	All tries in the 7 subsets	254,933	117,664	—	45.2%
NVIDIA Open-SWE-Traces	Outcome known (resolved is 0 or 1)	212,222	99,018	42,711	45.2%
NVIDIA Open-SWE-Traces	Same issue text as the most common one	212,222	99,018	0	45.2%

Table 4: All secondary estimates (fair, within-issue), with Benjamini–Hochberg q-values. Points per 100 attempts unless the outcome is a count.

ID	Question	AI	Estimate (95% CI)	Groups	q	Sig.
S1	Check your fix	Llama-70B (older AI)	+2.7 [+1.9, +3.6] pts	4,669	0.007	yes
S1	Check your fix	DeepSeek-V4-Flash / OpenHands / scale-swe	-0.7 [-2.2, +0.8] pts	1,011	0.797	
S1	Check your fix	MiniMax-M2.5 / OpenHands / swe-rebench-v2	+0.4 [-0.5, +1.4] pts	2,710	0.797	
S1	Check your fix	Qwen3.5-122B / OpenHands / swe-rebench-v2	+4.1 [+2.4, +5.9] pts	1,508	0.007	yes
S1	Check your fix	MiniMax-M2.5 / SWE-agent / swe-rebench-v2	+0.4 [-0.6, +1.4] pts	3,264	0.797	
S1	Check your fix	Qwen3.5-122B / SWE-agent / swe-rebench-v2	-1.2 [-3.0, +0.6] pts	1,145	0.695	
S1	Check your fix	Qwen3.6-27B / SWE-agent / scale-swe	+1.7 [-1.8, +5.3] pts	231	0.797	
S1	Check your fix	Qwen3.6-27B / SWE-agent / swe-rebench-v2	+0.1 [-1.5, +1.7] pts	1,503	1.000	
S2	Test suite after fix	Llama-70B (older AI)	-0.4 [-2.1, +1.1] pts	1,388	0.847	
S2	Test suite after fix	DeepSeek-V4-Flash / OpenHands / scale-swe	-0.2 [-2.4, +2.0] pts	507	1.000	
S2	Test suite after fix	MiniMax-M2.5 / OpenHands / swe-rebench-v2	-0.2 [-3.8, +3.4] pts	230	1.000	
S2	Test suite after fix	Qwen3.5-122B / OpenHands / swe-rebench-v2	+2.2 [-2.4, +7.3] pts	174	0.797	
S2	Test suite after fix	MiniMax-M2.5 / SWE-agent / swe-rebench-v2	+1.7 [-0.9, +4.6] pts	474	0.695	
S2	Test suite after fix	Qwen3.5-122B / SWE-agent / swe-rebench-v2	-1.2 [-6.8, +4.3] pts	171	0.977	
S2	Test suite after fix	Qwen3.6-27B / SWE-agent / scale-swe	-0.2 [-3.9, +3.3] pts	235	1.000	
S2	Test suite after fix	Qwen3.6-27B / SWE-agent / swe-rebench-v2	+0.8 [-1.4, +3.1] pts	631	0.797	
S3	Run any code after fix	DeepSeek-V4-Flash / OpenHands / scale-swe	-0.8 [-6.2, +4.7] pts	129	1.000	
S3	Run any code after fix	MiniMax-M2.5 / OpenHands / swe-rebench-v2	+1.1 [-11.2, +13.8] pts	24	1.000	underpowered
S3	Run any code after fix	Qwen3.5-122B / OpenHands / swe-rebench-v2	+0.0 [+0.0, +0.0] pts	8	1.000	underpowered
S3	Run any code after fix	MiniMax-M2.5 / SWE-agent / swe-rebench-v2	-0.7 [-5.7, +5.2] pts	38	1.000	underpowered
S3	Run any code after fix	Qwen3.5-122B / SWE-agent / swe-rebench-v2	+0.0 [+0.0, +0.0] pts	12	1.000	underpowered
S3	Run any code after fix	Qwen3.6-27B / SWE-agent / scale-swe	+5.6 [-5.6, +19.4] pts	19	0.847	underpowered
S3	Run any code after fix	Qwen3.6-27B / SWE-agent / swe-rebench-v2	+8.3 [-3.4, +23.4] pts	27	0.745	underpowered
S4	Extra tests after fix	DeepSeek-V4-Flash / OpenHands / scale-swe	+0.0 [-1.2, +1.1] pts	2,279	1.000	
S4	Extra tests after fix	MiniMax-M2.5 / OpenHands / swe-rebench-v2	+0.1 [-0.7, +0.9] pts	4,999	1.000	
S4	Extra tests after fix	Qwen3.5-122B / OpenHands / swe-rebench-v2	+3.2 [-1.6, +7.8] pts	156	0.695	
S4	Extra tests after fix	MiniMax-M2.5 / SWE-agent / swe-rebench-v2	+0.6 [-0.2, +1.4] pts	5,514	0.593	
S4	Extra tests after fix	Qwen3.5-122B / SWE-agent / swe-rebench-v2	-1.3 [-3.7, +1.2] pts	572	0.797	
S4	Extra tests after fix	Qwen3.6-27B / SWE-agent / scale-swe	-3.7 [-9.4, +1.6] pts	105	0.695	
S4	Extra tests after fix	Qwen3.6-27B / SWE-agent / swe-rebench-v2	-2.9 [-8.8, +2.4] pts	130	0.797	
S5	Clean up created files	DeepSeek-V4-Flash / OpenHands / scale-swe	+0.3 [-0.7, +1.3] pts	2,324	0.940	
S5	Clean up created files	MiniMax-M2.5 / OpenHands / swe-rebench-v2	+0.2 [-0.7, +1.2] pts	3,281	0.856	
S5	Clean up created files	Qwen3.5-122B / OpenHands / swe-rebench-v2	+1.8 [+0.9, +2.8] pts	4,032	0.007	yes
S5	Clean up created files	MiniMax-M2.5 / SWE-agent / swe-rebench-v2	+0.2 [-0.6, +1.1] pts	3,899	0.912	
S5	Clean up created files	Qwen3.5-122B / SWE-agent / swe-rebench-v2	+1.1 [-1.6, +3.9] pts	634	0.797	
S5	Clean up created files	Qwen3.6-27B / SWE-agent / scale-swe	-6.3 [-12.4, -0.9] pts	85	0.122	underpowered
S5	Clean up created files	Qwen3.6-27B / SWE-agent / swe-rebench-v2	+1.0 [-1.5, +3.5] pts	322	0.801	
S6	Test suite before fix	DeepSeek-V4-Flash / OpenHands / scale-swe	+0.7 [-0.4, +1.9] pts	1,854	0.797	
S6	Test suite before fix	MiniMax-M2.5 / OpenHands / swe-rebench-v2	-0.6 [-1.6, +0.3] pts	3,353	0.720	
S6	Test suite before fix	Qwen3.5-122B / OpenHands / swe-rebench-v2	-3.6 [-4.8, -2.3] pts	2,807	0.007	yes
S6	Test suite before fix	MiniMax-M2.5 / SWE-agent / swe-rebench-v2	-0.4 [-1.4, +0.6] pts	3,440	0.797	
S6	Test suite before fix	Qwen3.5-122B / SWE-agent / swe-rebench-v2	+0.2 [-1.5, +1.8] pts	1,486	1.000	
S6	Test suite before fix	Qwen3.6-27B / SWE-agent / scale-swe	+0.4 [-0.4, +1.1] pts	3,367	0.797	
S6	Test suite before fix	Qwen3.6-27B / SWE-agent / swe-rebench-v2	+0.2 [-0.8, +1.3] pts	2,888	1.000	
S7	Cost: steps	Llama-70B (older AI)	+10.0 [+8.8, +11.3]	1,870	0.007	yes
S7	Cost: steps	Newer AIs, pooled	+1.9 [+1.4, +2.3]	9,776	0.007	yes
S7	Cost: characters read (thousands)	Newer AIs, pooled	+3.4 [+2.4, +4.4]	9,776	0.007	yes
S7	Cost: characters written (thousands)	Newer AIs, pooled	+1.5 [+0.9, +2.0]	9,776	0.007	yes
S9	Test first, language = go	Newer AIs, pooled	+0.1 [-1.3, +1.6] pts	1,730	1.000	
S9	Test first, language = java	Newer AIs, pooled	+1.5 [-1.4, +4.5] pts	403	0.797	
S9	Test first, language = javascript	Newer AIs, pooled	+0.5 [-1.0, +2.1] pts	1,171	0.801	
S9	Test first, language = php	Newer AIs, pooled	+2.5 [-0.8, +5.6] pts	347	0.593	
S9	Test first, language = python	Newer AIs, pooled	-0.1 [-0.9, +0.7] pts	3,851	1.000	
S9	Test first, language = rust	Newer AIs, pooled	-1.1 [-3.1, +0.8] pts	750	0.797	
S9	Test first, language = typescript	Newer AIs, pooled	-0.1 [-1.6, +1.6] pts	1,486	1.000	
S10	Submitted because the context ran out	Llama-70B (older AI)	+11.0 [+9.2, +12.8] pts	1,870	0.007	yes
S10	Any context-limit exit	Llama-70B (older AI)	+11.0 [+9.1, +12.8] pts	1,870	0.007	yes
S12	First move think vs view (Qwen3-Coder)	Qwen3-Coder	-0.1 [-0.7, +0.6] pts	2,067	1.000	
S13	First move find_file vs search_dir	Llama-70B (older AI)	-0.5 [-2.0, +1.0] pts	1,485	0.801	
S13	First move ls vs search_dir	Llama-70B (older AI)	-1.2 [-2.5, +0.1] pts	1,697	0.354	
S13	First move open vs search_dir	Llama-70B (older AI)	+5.0 [-1.1, +11.3] pts	136	0.593	
S13	First move grep vs search_dir	Llama-70B (older AI)	-1.2 [-4.0, +1.4] pts	395	0.797	
S8	Reproduce first x issue type: has_code_block	Llama-70B (older AI)	+2.6 [-1.0, +6.2] pts	—	0.680	
S8	Reproduce first x issue type: has_stack_trace	Llama-70B (older AI)	-1.9 [-6.3, +2.5] pts	—	0.797	
S8	Reproduce first x issue type: names_file	Llama-70B (older AI)	-0.8 [-4.5, +2.9] pts	—	0.953	
S8	Reproduce first x issue type: long_issue	Llama-70B (older AI)	+0.2 [-3.8, +4.1] pts	—	1.000	
S9	Test first x language (vs python): lang_go	Newer AIs, pooled	+0.2 [-1.9, +2.3] pts	—	1.000	
S9	Test first x language (vs python): lang_java	Newer AIs, pooled	+1.6 [-2.3, +5.5] pts	—	0.797	
S9	Test first x language (vs python): lang_javascript	Newer AIs, pooled	+0.7 [-1.5, +2.9] pts	—	0.847	
S9	Test first x language (vs python): lang_php	Newer AIs, pooled	+2.6 [-1.6, +6.8] pts	—	0.731	
S9	Test first x language (vs python): lang_rust	Newer AIs, pooled	-1.0 [-3.8, +1.8] pts	—	0.801	
S9	Test first x language (vs python): lang_typescript	Newer AIs, pooled	+0.0 [-2.2, +2.3] pts	—	1.000	
S11	Finding the right file: per-step hazard	Llama-70B (older AI)	-12.2 [-13.8, -10.7] pts	—	<0.001	yes